01 / Gateway & Runtime implementation guide

A reusable foundation for a standalone CRM
SaaS14 source review | 22 October 2026

Deploy the existing Gateway and Runtime images in a separate project. SaaS, ERP, platform user management and platform tenant management are not required. This guide includes a complete local HTTP deployment, HTTPS configuration, publishing, and optional Broker and SMTP services.

Architecture

```
Browser
  | HTTP / HTTPS
Gateway (published ports)
  | internal HTTP
CRM Runtime: public/ + api/ + config/
  | optional server-to-server calls
Broker -&gt; PostgreSQL        SMTP -&gt; mail provider
```

Gateway: routes requests by URL prefix, terminates TLS, forwards request metadata, issues a visitor cookie and writes logs. It does not authenticate users.

Runtime: serves static files and maps API URLs to CommonJS handler files. Each container mounts its own content at /app/content. Built-in technical packages include pg and nodemailer.

Broker and SMTP: optional content packages running in additional Runtime containers. Their source folders are not included in the Runtime image.

Release names used in this guide

```
opnelcom/core-gateway:1.0.0
opnelcom/core-runtime:2.0.0
```

These are intended publication names, not confirmation that the images exist on Docker Hub. Examples use port 8080 for local HTTP and crm.example.com as a domain placeholder. Replace all example credentials before deployment.

Validation boundary: examples follow the repository source contract. This document does not represent a completed Docker build, image publication or live deployment test.

02 / Build and publish the images

Prerequisites: Docker Engine or Docker Desktop using Linux containers, a Docker Hub account, and access to the SaaS14 image sources. Create core-gateway and core-runtime repositories under opnelcom and explicitly select their visibility.

Run in PowerShell from the source repository

```
Set-Location Y:\projects\saas14
docker login
docker build -t opnelcom/core-gateway:1.0.0 ./Applications/Core-Gateway
docker build -t opnelcom/core-runtime:2.0.0 ./Applications/Core-Runtime
docker push opnelcom/core-gateway:1.0.0
docker push opnelcom/core-runtime:2.0.0
```

Check that each command succeeds before continuing. In Docker Hub, confirm both tags appear. Private repositories require docker login on deployment machines as well.

What is published

Gateway copies package.json and server.js. Runtime installs its locked dependencies and copies server.js, config.js and healthcheck.js. Neither Dockerfile copies the Volumes folder, application code, certificates or deployment credentials.

Version discipline

Publish a new version when image code or dependencies change. Keep released tags stable and reference explicit versions in Compose. For stricter reproducibility, deploy the published image digest. Updating mounted CRM content does not require rebuilding Runtime unless its technical dependencies change.

Target CPU architecture

The standard build above targets the builder platform. If deployment includes both x86-64 and ARM64, use a builder that supports both, and publish a multi-platform image:

```
docker buildx build --platform linux/amd64,linux/arm64 `
  -t opnelcom/core-runtime:2.0.0 --push ./Applications/Core-Runtime
```

Use the equivalent Gateway context and tag for Gateway. Both current Dockerfiles use node:20-alpine; assess base-image support and dependency updates as part of release maintenance before production use.

03 / Create the standalone project

Create a new project directory. All paths below are relative to that directory. The new project consumes the published images; it needs no image build folders.

```
crm-project/
  compose.yaml
  Volumes/
    CRM/
      config/config.json
      api/ping.js
      api/_shared/       # private reusable helpers
      public/index.html
      logs/
    Gateway/
      config/gateway.json
      logs/
      certs/            # needed when TLS is enabled
```

Volumes/CRM/config/config.json

```
{
  "name": "crm",
  "port": 3000,
  "maxBodyBytes": 1048576,
  "logToFile": true,
  "fileLogMode": "failures"
}
```

Volumes/Gateway/config/gateway.json - local HTTP

```
{
  "hostname": "localhost",
  "port": 3000,
  "tls": { "enabled": false },
  "routes": [
    {
      "prefix": "/",
      "target": "http://crm:3000",
      "stripPrefix": false
    }
  ]
}
```

The Gateway target crm is the Compose service name. Incoming /api/ping reaches /api/ping in CRM. For a /crm prefix, use prefix /crm and stripPrefix true; ensure browser asset and API URLs include /crm. Gateway selects the longest matching route prefix.

Runtime reads its JSON at process startup. Restart its container after configuration changes. Production handler modules are cached; restart CRM after API code changes. Gateway reads routing configuration on each request, but listener ports and TLS files are loaded at startup.

04 / Complete local Compose deployment

Save as compose.yaml. Create the folders and configuration files from page 3 and the application files on page 5 before starting. Only Gateway publishes a host port.

```
name: crm-project
services:
  crm:
    image: opnelcom/core-runtime:2.0.0
    restart: unless-stopped
    init: true
    volumes:
      - ./Volumes/CRM:/app/content:ro
      - ./Volumes/CRM/logs:/app/content/logs
    networks: [application]
    healthcheck:
      test: ["CMD", "node", "/app/healthcheck.js"]
      interval: 30s
      timeout: 5s
      retries: 5
      start_period: 15s
    security_opt: ["no-new-privileges:true"]

  gateway:
    image: opnelcom/core-gateway:1.0.0
    restart: unless-stopped
    init: true
    ports: ["8080:3000"]
    volumes:
      - ./Volumes/Gateway:/app/content:ro
      - ./Volumes/Gateway/logs:/app/content/logs
    networks: [frontend, application]
    depends_on:
      crm: {condition: service_healthy}
    security_opt: ["no-new-privileges:true"]

networks:
  frontend:
    driver: bridge
  application:
    driver: bridge
    internal: true

docker compose config --quiet
docker compose pull
docker compose up -d
docker compose ps
Invoke-RestMethod http://localhost:8080/api/ping
```

Open http://localhost:8080/ for the UI. Runtime health checks confirm process health, not database or mail delivery. /health through Gateway checks Gateway itself; /api/ping also verifies proxying to CRM.

05 / Implement reusable CRM content

Volumes/CRM/public/index.html

```
&lt;!doctype html&gt;
&lt;html lang="en"&gt;
&lt;meta charset="utf-8"&gt;
&lt;meta name="viewport" content="width=device-width, initial-scale=1"&gt;
&lt;title&gt;CRM&lt;/title&gt;
&lt;h1&gt;CRM&lt;/h1&gt;
&lt;p id="status"&gt;Connecting...&lt;/p&gt;
&lt;script&gt;
fetch('/api/ping')
  .then(r =&gt; { if (!r.ok) throw Error('Unavailable'); return r.json(); })
  .then(x =&gt; { document.querySelector('#status').textContent = x.message; })
  .catch(() =&gt; { document.querySelector('#status').textContent = 'Unavailable'; });
&lt;/script&gt;
&lt;/html&gt;
```

Volumes/CRM/api/ping.js

```
'use strict';
module.exports = async ctx =&gt; {
  if (ctx.req.method !== 'GET') {
    return ctx.send(405, {error: 'GET required'}, {allow: 'GET'});
  }
  return {message: 'CRM Runtime is ready', requestId: ctx.requestId};
};
```

Runtime handler contract

A handler receives ctx and returns JSON, calls ctx.send(status, body, headers), or writes to ctx.res. Available values include req, res, url, query, cookies, body, config, logger, crypto, requestId, setCookie and clearCookie.

For example, /api/customers/list maps to api/customers/list.js. Files or folders beginning with an underscore are private to HTTP routing, making api/_shared suitable for reusable validation, service clients and permission checks.

Runtime parses JSON and URL-encoded bodies. Other content is passed as a UTF-8 raw string; binary or multipart uploads need deliberate application handling. Missing static paths fall back to public/index.html.

Application ownership

Implement login, session verification, authorisation, CRM rules and any tenant filtering in the CRM application. Runtime does not automatically apply middleware. Reusable wrappers can explicitly protect each API handler. Never treat the Gateway core_visitor cookie as proof of identity.

Runtime does not install mounted application dependencies. Install additional Linux-compatible packages into /app/content/node_modules during deployment, using the application package manifest and lockfile. Avoid copying Windows native modules into Linux containers.

06 / Enable HTTPS and configure routing

Point your domain to the deployment host and obtain a valid certificate for it. Certificate issuance and renewal are separate operational tasks; Gateway can serve ACME challenge files but does not request or renew certificates itself.

Replace the Gateway JSON for production

```
{
  "hostname": "crm.example.com",
  "port": 3000,
  "tls": {
    "enabled": true,
    "port": 3443,
    "certFile": "certs/fullchain.pem",
    "keyFile": "certs/privkey.pem",
    "redirectHttpToHttps": true,
    "hsts": true
  },
  "acme": {"challengeRoot": "acme/.well-known/acme-challenge"},
  "routes": [
    {"prefix": "/", "target": "http://crm:3000", "stripPrefix": false}
  ]
}
```

Place fullchain.pem and privkey.pem in Volumes/Gateway/certs. Keep the private key out of source control and Docker Hub. Ensure the Gateway process can read the files.

Change Gateway ports in the new Compose file

```
ports:
  - "80:3000"
  - "443:3443"
```

Allow inbound TCP 80 and 443. The existing redirect implementation constructs an HTTPS URL without a custom port; the standard external HTTPS port 443 fits this configuration.

```
docker compose up -d
docker compose restart gateway
Invoke-RestMethod https://crm.example.com/api/ping
```

Restart Gateway after replacing certificates to reload TLS. Use HSTS only after HTTPS is working reliably. Route targets currently support internal http: URLs, not https: upstreams.

Multiple applications

Additional Runtime services can share this Gateway using prefixes such as /crm and /reports. Each receives its own content volume and configuration. Update both Compose service definitions and Gateway routes together. Do not route Broker or SMTP to the public browser interface.

07 / Optional Broker and SMTP services

Copy the reusable Core-Broker and Core-SMTP content folders into the new project. Retain their api and package manifests, but provide fresh configuration. Do not copy existing credentials, logs, database profiles or certificates. Both run on opnelcom/core-runtime:2.0.0.

Add service definitions under services in Compose

```
  broker:
    image: opnelcom/core-runtime:2.0.0
    restart: unless-stopped
    volumes:
      - ./Volumes/Core-Broker:/app/content:ro
      - ./Volumes/Core-Broker/logs:/app/content/logs
    networks: [application, database]
  smtp:
    image: opnelcom/core-runtime:2.0.0
    restart: unless-stopped
    volumes:
      - ./Volumes/Core-SMTP:/app/content:ro
      - ./Volumes/Core-SMTP/logs:/app/content/logs
    networks: [application, outbound]
  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: crm
      POSTGRES_USER: crm_user
      POSTGRES_PASSWORD: ${CRM_DB_PASSWORD:?Set CRM_DB_PASSWORD}
    volumes: ["crm-db:/var/lib/postgresql/data"]
    networks: [database]
```

Add these entries under the existing networks section, and volumes at the top level. SMTP needs outbound access to its provider; PostgreSQL does not need a published host port.

```
  database:
    driver: bridge
    internal: true
  outbound:
    driver: bridge

volumes:
  crm-db:
```

Set CRM_DB_PASSWORD in a local .env file excluded from source control. Broker configuration must contain the matching database password; its JSON is not automatically interpolated from Compose environment variables. Provision database tables and migrations through CRM-owned scripts.

Apply the Runtime health check, init and security settings from page 4 to these Runtime services as appropriate. Configure startup readiness and retry transient database connections in the application.

08 / Profiles and internal service calls

Volumes/Core-Broker/config/config.json

```
{
  "name": "crm-broker", "port": 3000,
  "dbProfiles": [{
    "name": "crm", "profileKey": "REPLACE_BROKER_KEY",
    "host": "db", "port": 5432, "database": "crm",
    "user": "crm_user", "password": "MATCH_CRM_DB_PASSWORD"
  }]
}
```

Volumes/Core-SMTP/config/config.json

```
{
  "name": "crm-smtp", "port": 3000,
  "smtpProfiles": [{
    "name": "crm", "profileKey": "REPLACE_SMTP_KEY",
    "host": "smtp.example.com", "port": 587, "secure": false,
    "tlsRejectUnauthorized": true,
    "user": "crm@example.com", "password": "REPLACE_SMTP_PASSWORD",
    "from": "crm@example.com"
  }]
}
```

Use independent random service keys. Add application-owned broker and mail settings to CRM config:

```
"broker": {"url": "http://broker:3000/api/broker/query",
           "profile": "crm", "key": "REPLACE_BROKER_KEY"},
"mail": {"url": "http://smtp:3000/api/mail/send",
         "profile": "crm", "key": "REPLACE_SMTP_KEY"}
```

CRM server-side call pattern

```
const b = ctx.config.broker;
const response = await fetch(b.url, {
  method: 'POST',
  headers: {'content-type': 'application/json', 'x-core-key': b.key},
  signal: AbortSignal.timeout(10000),
  body: JSON.stringify({brokerProfile: b.profile,
    text: 'SELECT $1::text AS message', values: ['CRM connected']})
});
if (!response.ok) throw Error('Database service unavailable');
const result = await response.json();
```

Put shared clients in CRM api/_shared and call them from authenticated handlers. Use parameterised SQL and enforce access rules before queries. Broker permits SQL within the database role privileges; it does not enforce CRM user permissions. For transactions, POST brokerProfile and statements to /api/broker/transaction. Mail uses profile, to, subject, text and/or html at /api/mail/send, with its own x-core-key header.

09 / Operate, troubleshoot and maintain

Routine commands

```
docker compose ps
docker compose logs --tail 100 gateway crm
docker compose restart crm
docker compose pull
docker compose up -d
```

After changing an image version in Compose, pull and recreate the affected service. After changing mounted API code or Runtime configuration, restart that Runtime service. Keep a previous image version and a matching content release for rollback; database migrations need their own recovery plan.

Symptoms and checks

Image pull denied: verify the opnelcom repository, version tag, visibility and docker login permissions.

Runtime exits: verify /app/content/config/config.json exists and port is an integer. Environment variables such as PORT and CONTENT_ROOT do not override the current fixed contract.

Gateway 502: check target service name, target port, shared network and CRM logs. A Gateway /health response alone does not prove CRM is reachable.

API 404: confirm api/path.js exports a function and the requested path is /api/path. Underscore-prefixed paths cannot be called directly.

TLS fails: verify hostname, certificate chain, key permissions, published ports and Gateway restart after certificate replacement.

Broker or SMTP 403: check the selected profile and x-core-key. Keep credentials server-side. A missing SMTP host returns development acceptance without sending mail.

Production ownership

Back up PostgreSQL and application uploads, protect configuration and TLS keys, rotate credentials and retain logs appropriately. Gateway visitor logs include client metadata; define access and retention rules. Use unique Broker keys per profile: its profiles endpoint accepts any configured profile key and lists metadata for all profiles.

The base example is a deployment starting point. Implement CRM session security, authorisation, input validation, rate controls and operational monitoring according to application requirements. Keep business rules in CRM content rather than Gateway, Runtime, Broker or SMTP.

Source basis and further reading

Repository basis: Applications/Core-Gateway/{Dockerfile,server.js}; Applications/Core-Runtime/{Dockerfile,README.md,server.js,config.js,healthcheck.js}; Volumes/Core-Broker/api; Volumes/Core-SMTP/api/mail/send.js; compose.yaml. Reviewed for this guide on 22 October 2026.

Docker image publishing: docs.docker.com/docker-hub/repos/manage/hub-images/push/
Multi-platform builds: docs.docker.com/build/building/multi-platform/